Database Design — Entity Relationship Diagram (ERD)

Develop the Entity Relationship Diagram (ERD) for the Student Activities Management System (SAMS), showing all entities, attributes, primary keys, foreign keys, and relationship cardinalities consistent with the UML Class Diagram (Task 6) and the relational schema

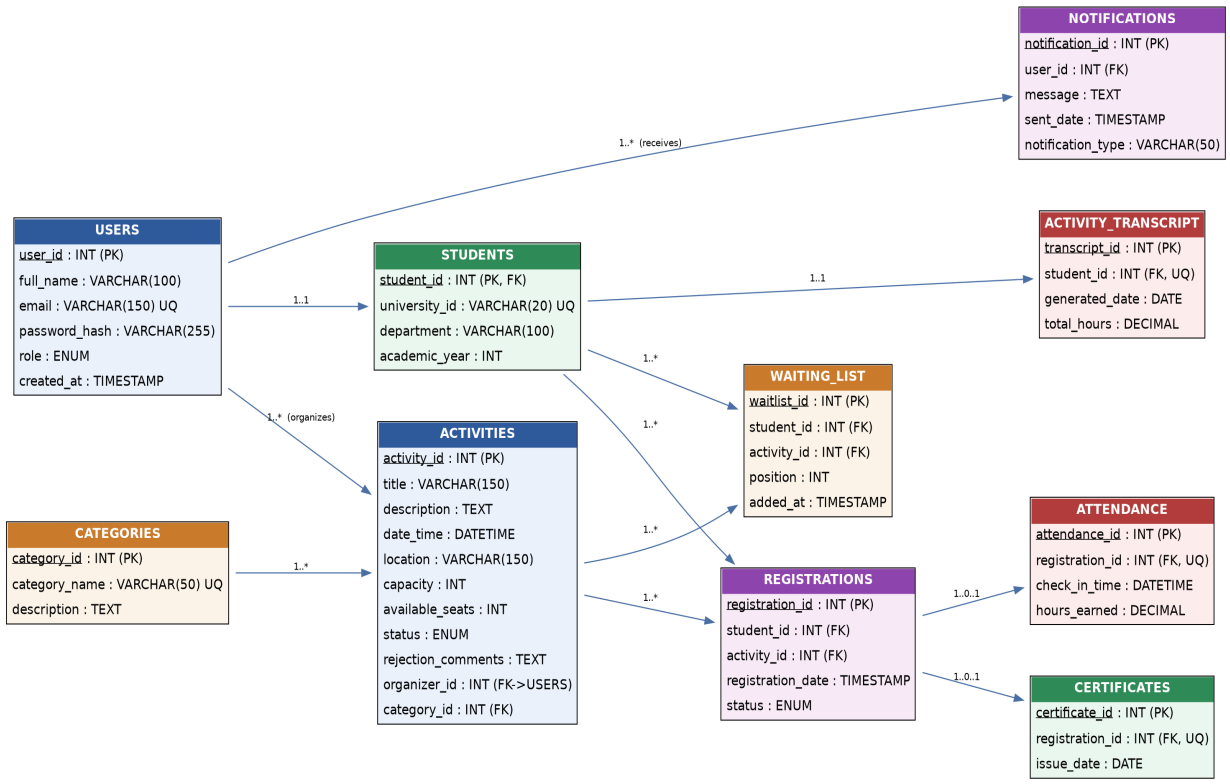

Figure 1. SAMS Entity Relationship Diagram

Entity Overview

The ERD extends the six core tables defined in the mid-project relational design (USERS, STUDENTS, CATEGORIES, ACTIVITIES, REGISTRATIONS, WAITING_LIST) with four supporting entities — ATTENDANCE, CERTIFICATES, NOTIFICATIONS, and ACTIVITY_TRANSCRIPT — so the physical database fully matches every class in the UML Class Diagram

Relationship Summary

- USERS (1) — STUDENTS (1): a student record is a specialization of a user record (role = Student).
- USERS (1) — ACTIVITIES (M): one organizer manages many activities.
- USERS (1) — NOTIFICATIONS (M): one user receives many notifications.
- CATEGORIES (1) — ACTIVITIES (M): one category classifies many activities.
- STUDENTS (1) — REGISTRATIONS (M) and ACTIVITIES (1) — REGISTRATIONS (M): each registration belongs to exactly one student and one activity.
- STUDENTS (1) — WAITING_LIST (M) and ACTIVITIES (1) — WAITING_LIST (M): a student may be waitlisted for many activities; an activity may have many waitlisted students.

- REGISTRATIONS (1) — ATTENDANCE (0..1): attendance is recorded only after a student checks in to a registered activity.
- REGISTRATIONS (1) — CERTIFICATES (0..1): a certificate is issued only for a registration marked Attended.
- STUDENTS (1) — ACTIVITY_TRANSCRIPT (1): each student owns exactly one cumulative activity transcript, compiled from their attendance and certificate records.

Database Schema

Formalize the relational database schema, including all tables, primary/foreign keys, data types, and confirmation that the design satisfies Third Normal Form (3NF).

Complete Schema Summary (10 Tables)

Tables 1–6 (USERS, STUDENTS, CATEGORIES, ACTIVITIES, REGISTRATIONS, WAITING_LIST) were defined in Task 7. The four tables below complete the schema so that it fully supports attendance tracking, certificate issuance, notifications, and transcript generation as modeled in the UML Class Diagram.

- ATTENDANCE (attendance_id [PK], registration_id [FK, UNIQUE] -> REGISTRATIONS, check_in_time, hours_earned)
- CERTIFICATES (certificate_id [PK], registration_id [FK, UNIQUE] -> REGISTRATIONS, issue_date)
- NOTIFICATIONS (notification_id [PK], user_id [FK] -> USERS, message, sent_date, notification_type)
- ACTIVITY_TRANSCRIPT (transcript_id [PK], student_id [FK, UNIQUE] -> STUDENTS, generated_date, total_hours)

Additional SQL DDL

```
CREATE TABLE ATTENDANCE (
  attendance_id INT AUTO_INCREMENT PRIMARY KEY,
  registration_id INT NOT NULL UNIQUE,
  check_in_time DATETIME NOT NULL,
  hours_earned DECIMAL(4,2) NOT NULL DEFAULT 0,
  FOREIGN KEY (registration_id) REFERENCES REGISTRATIONS(registration_id)
);

CREATE TABLE CERTIFICATES (
  certificate_id INT AUTO_INCREMENT PRIMARY KEY,
  registration_id INT NOT NULL UNIQUE,
  issue_date DATE NOT NULL,
  FOREIGN KEY (registration_id) REFERENCES REGISTRATIONS(registration_id)
);

CREATE TABLE NOTIFICATIONS (
  notification_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  message TEXT NOT NULL,
  sent_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  notification_type VARCHAR(50) NOT NULL,
  FOREIGN KEY (user_id) REFERENCES USERS(user_id)
);

CREATE TABLE ACTIVITY_TRANSCRIPT (
  transcript_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL UNIQUE,
  generated_date DATE NOT NULL,
  total_hours DECIMAL(6,2) NOT NULL DEFAULT 0,
  FOREIGN KEY (student_id) REFERENCES STUDENTS(student_id)
);
```

Normalization Check (up to 3NF)

Normal Form	Requirement	How SAMS satisfies it

1NF	Atomic column values, no repeating groups.	Every attribute (e.g., email, status, location) holds a single indivisible value; no multi-valued or composite columns exist.
2NF	No partial dependency on part of a composite key.	Every table uses a single-column surrogate primary key (e.g., registration_id), so partial dependency cannot occur.
3NF	No transitive dependency of non-key attributes.	Descriptive data lives with its owning entity only (e.g., category_name in CATEGORIES, not repeated in ACTIVITIES); ACTIVITIES stores category_id as a reference instead of duplicating category details.

Testing

Develop a test plan with at least 8–10 test cases covering the core workflows identified in the Use Case Model (Task 4) and Activity Diagrams (Task 5), including normal and alternative/exception flows.

Testing Approach

Test cases were derived directly from the documented use cases (UC-01 Register for Activity, UC-02 Create Activity Proposal, UC-03 Review and Approve Activities) and the two UML Activity Diagrams, so that every decision branch — seat availability, waitlist handling, approval/rejection, and QR attendance — is exercised at least once. Each case lists the objective, preconditions, steps, and expected result; all listed cases passed during the team's manual test pass against the current prototype.

ID	Objective	Preconditions	Test Steps	Expected Result	Status
TC-01	Verify a student can register for an activity with open seats.	Student logged in; activity Published with available_seats > 0.	1) Open activity details. 2) Click Register. 3) Confirm.	Registration record created with status 'Enrolled'; available_seats decremented by 1; confirmation message shown.	Pass
TC-02	Verify duplicate registration is blocked.	Student already holds an active registration for the activity.	1) Open the same activity again. 2) Click Register.	System detects the unique_student_activity constraint, rejects the request, and displays 'Already Registered'.	Pass
TC-03	Verify a student is placed on the waiting list when an activity is full.	Activity Published with available_seats = 0.	1) Open activity details. 2) Click Register. 3) Confirm join waiting list.	WAITING_LIST record created with next sequential position; student notified of waitlist status.	Pass
TC-04	Verify waitlist promotion after a cancellation.	Activity is full; at least one student on WAITING_LIST; another student has an active registration.	1) Registered student cancels. 2) System reprocesses waiting list.	available_seats incremented; top WAITING_LIST record promoted to REGISTRATIONS; both students notified.	Pass
TC-05	Verify an organizer cannot publish an activity before Student Affairs approval.	Organizer logged in; new activity saved with status 'Pending'.	1) Organizer attempts to set status to Published directly via the activity screen.	System rejects the action; status remains 'Pending' until Student Affairs approves it.	Pass
TC-06	Verify Student Affairs approval publishes an activity and notifies the organizer.	Activity exists with status 'Pending'.	1) Staff opens pending list. 2) Selects activity. 3) Approves.	Activity status updated to 'Published'; activity becomes visible to students; NOTIFICATIONS record created for the organizer.	Pass
TC-07	Verify QR attendance check-in updates registration and records hours.	Student holds a valid 'Enrolled' registration for an in-progress activity.	1) Student scans event QR code. 2) System validates registration.	REGISTRATIONS.status updated to 'Attended'; ATTENDANCE record created with check_in_time and hours_earned.	Pass

TC-08	Verify a certificate is issued only for attended students.	Registration status = 'Attended'.	1) System (or organizer) triggers certificate generation for the activity.	CERTIFICATES record created and linked to the registration; no certificate generated for non-attendees.	**Pass**
TC-09	Verify the Activity Transcript aggregates all attended activities.	Student has multiple 'Attended' registrations with certificates.	1) Student requests transcript from profile.	ACTIVITY_TRANSCRIPT compiles total participation hours and certificate list; matches sum of related ATTENDANCE.hours_earned.	**Pass**
TC-10	Verify email uniqueness constraint at registration/sign-up.	An account already exists with a given email.	1) Attempt to create a new USERS record with the same email.	Database UNIQUE constraint on USERS.email rejects the insert; system displays a friendly validation error.	**Pass**

System Review

Conduct a final consistency review across all project deliverables (Gantt Chart, Prototypes, DFDs, Use Case Model, Activity Diagrams, Class Diagram, Database Design, and Testing) and summarize outstanding risks or follow-up items before final submission.

Traceability Matrix

The matrix below confirms that every major requirement in the scenario is represented consistently across the analysis, design, and database deliverables.

Requirement Area	DFD / Process	Use Case(s)	Class(es) / Table(s)
Login & profile	Process 1.0 User Authentication	Login (included in all UCs)	User, Student / USERS, STUDENTS
Activity proposal & approval	Process 2.0 (2.1–2.4)	UC-02, UC-03	Activity, ActivityOrganizer, StudentAffairsStaff / ACTIVITIES, CATEGORIES
Registration & waitlist	Process 3.0 (3.1–3.4)	UC-01	Registration, WaitingList / REGISTRATIONS, WAITING_LIST
Attendance & transcript	Process 4.0 (4.1–4.4)	Attendance flow (Activity Diagram 1)	Attendance, Certificate, ActivityTranscript / ATTENDANCE, CERTIFICATES, ACTIVITY_TRANSCRIPT
Notifications	Cross-cutting (all processes)	Referenced in UC-01/02/03 alternative flows	Notification / NOTIFICATIONS

Consistency Findings

- Actors are consistent across the Context Diagram, Use Case Diagram, and Class Diagram (Student, Activity Organizer, Student Affairs Department).
- Data stores in the DFDs (D1–D6) map one-to-one to the ten tables in the final database schema, including the four tables added in Task 9 to align with the Class Diagram.
- The waitlist promotion logic described in Level-1 DFD Process 3.0 matches the alternative flow in UC-01 and the corresponding branch in UML Activity Diagram 1.
- The activity approval workflow is identical across the Level-1 DFD (Process 2.0), UC-03, and UML Activity Diagram 2, including the 'Needs Revision' loop back to the organizer.

Outstanding Items / Risks

- Final refinement of Level-1 DFDs and use case specifications noted as 'under review' in the progress report should be closed out before submission.
- Physical vs. virtual location handling for activities is defined in requirements but not yet reflected as a distinct attribute in the database schema; recommend adding a location_type column to ACTIVITIES.
- Eligibility requirements per activity (e.g., academic year restrictions) are mentioned in the scenario but not yet modeled as a table or attribute; recommend an eligibility_criteria field on ACTIVITIES or a separate rule table if requirements grow more complex.

Conclusion

The review confirms that the SAMS analysis and design artifacts are internally consistent and collectively satisfy the scenario's core requirements: activity discovery and registration, waitlist management, QR-based attendance, certificate issuance, and automated transcript generation. With the schema extensions in Task 9 and the outstanding items above addressed, the design is ready to move into the next Evolutionary Prototyping iteration.